

SIMATS ENGINEERING ASSESSMENT TOOL 1

COURSE NAME: INTERNET PROGRAMMING

COURSE CODE: CSA4305

COURSE OUTCOME COVERED

CO 3: Implement dynamic web applications by utilizing DOM-based client-side scripting and employing Java Servlets for server-side request processing and response handling.

(L3) Assessment Tool : Scenario-Based Assessment

Weightage: 40%

QUESTIONS: 3 Total Marks: 30

Code of Conduct:

I N. Loka Bandhava (192324167) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



Question 1

Suppose that a college wants to develop an online student registration system where students submit their details through a web form, and the data is processed using a Java Servlet. Explain the architecture of a servlet-based application. Design and implement a servlet to handle form data using both GET and POST methods. Describe the servlet life cycle (init, service, destroy) and justify which method is more appropriate for handling sensitive data.

Answer to Question 1:

A) Architecture of a Servlet-Based Application:

A servlet-based web application follows a client-server architecture, typically implementing a Request-Response model:

1. Client (Browser): The user interacts with an HTML form and submits details. This triggers an HTTP request (GET or POST) containing user inputs.
2. Web Server & Servlet Container (e.g., Apache Tomcat): The server receives the HTTP request and forwards it to the Servlet Container. The container maps the URL to the corresponding servlet, manages the servlet instance life cycle, and encapsulates the HTTP request and response into

HttpServletRequest and HttpServletResponse objects.

3. Servlet: The servlet acts as the controller, executing database operations or business logic, and outputs HTML or JSON directly back to the client via the response object.

B) Servlet Implementation (StudentRegistrationServlet.java):

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class StudentRegistrationServlet extends HttpServlet {
    // Handles GET request (e.g., returning registration form details or simple
    searches)
    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
    throws ServletException, IOException {
        response.setContentType("text/html");
        PrintWriter out = response.getWriter();
        String regNo = request.getParameter("regNo");
        out.println("<html><body>");
        out.println("<h3>Student Info Requested via GET: " + regNo + "</h3>");
        out.println("</body></html>");
    }

    // Handles POST request (processing registration details)
    @Override
    protected void doPost(HttpServletRequest request, HttpServletResponse
    response)
    throws ServletException, IOException {
        response.setContentType("text/html");
        PrintWriter out = response.getWriter();
        String name = request.getParameter("studentName");
        String regNo = request.getParameter("regNo");
        String email = request.getParameter("email");

        // Database save logic goes here

        out.println("<html><body>");
        out.println("<h2>Registration Successful!</h2>");
        out.println("<p>Name: " + name + "</p>");
        out.println("<p>Reg No: " + regNo + "</p>");
        out.println("<p>Email: " + email + "</p>");
        out.println("</body></html>");
    }
}
```

C) Servlet Life Cycle (init, service, destroy):

1. init() Method: Invoked once by the servlet container when loading and instantiating the servlet class. It initializes resources (e.g., database connection setups).

2. service() Method: Invoked for every incoming request. It decodes the HTTP request type (GET, POST, etc.) and routes it to doGet(), doPost(), etc.
3. destroy() Method: Invoked once by the container before removing the servlet instance. It frees allocated resources.

D) HTTP Method Justification for Sensitive Data:

- Post Method is strictly recommended for handling sensitive data.
- Justification: GET appends form parameter values to the browser URL address bar in a clear-text query string. This exposes passwords and tokens in browser history, bookmarks, proxy logs, and server records. POST sends parameters inside the HTTP request body payload rather than the URL, securing data transmission (especially when combined with HTTPS encryption).

Question 2

Suppose that a website requires users to log in and maintain their session across multiple pages after authentication. Design a session management system using HttpSession and Cookies in Servlets. Explain how session tracking works in web applications and compare session-based tracking with cookies.

Answer to Question 2:

A) Session Management System Implementation:

Here is a login servlet checking credentials and establishing a session utilizing HttpSession and Cookie objects:

```
// LoginServlet.java
protected void doPost(HttpServletRequest request, HttpServletResponse response)
throws ServletException, IOException {
    String uname = request.getParameter("username");
    String pass = request.getParameter("password");

    if (uname.equals("admin") && pass.equals("secret")) {
        // 1. Establish HttpSession on server
        HttpSession session = request.getSession(true);
        session.setAttribute("user", uname);

        // 2. Add custom persistence Cookie
        Cookie userCookie = new Cookie("loggedInUser", uname);
        userCookie.setMaxAge(60 * 60); // 1 hour validity
        response.addCookie(userCookie);

        response.sendRedirect("dashboard");
    } else {
        response.sendRedirect("login.html");
    }
}
```

```
}  
}
```

B) How Session Tracking Works in Web Applications:

Since HTTP is a stateless protocol, each request is independent. Session tracking correlates requests from the same client:

1. Server creates a unique Session ID (JSESSIONID) when a new HttpSession is requested.
2. Server returns this Session ID to the client's browser (normally via a temporary session cookie).
3. For subsequent requests, the browser sends the Session ID back in the Cookie header.
4. The server checks this ID against its internal session map to retrieve the corresponding session context.

C) Comparison: Session-based Tracking vs. Cookies:

Feature	Session-based Tracking (HttpSession)	Cookie-based Tracking
Storage Location	Stored on the Web Server memory.	Stored locally on the client's browser.
Security Level	High (sensitive data is stored securely on the server side).	Low (cookies can be read, tampered, or intercepted).
Storage Capacity	Unlimited (limited only by server memory limits).	Limited (maximum 4KB per cookie).
Browser Settings	Works using URL rewriting if cookies are disabled.	Fails completely if browser disables cookie permissions.

Question 3

Suppose that an e-commerce platform needs to store and retrieve product details from a database using a Java-based web application. Design a JDBC-based application to connect to a database, insert product data, and retrieve it using SQL queries. Explain the JDBC architecture and describe the steps involved in database connectivity along with proper exception handling.

Answer to Question 3:

A) JDBC Implementation (ProductDAO.java):

Here is a DAO implementation demonstrating database connection, product insertion, and selection:

```
import java.sql.*;  
  
public class ProductDAO {  
    private static final String URL = "jdbc:mysql://localhost:3306/ecommerce_db";  
    private static final String USER = "root";
```

```

private static final String PASS = "root123";

// Inserts a new product
public void insertProduct(int id, String name, double price) {
    String query = "INSERT INTO products (id, name, price) VALUES (?, ?, ?)"; try
(Connection conn = DriverManager.getConnection(URL, USER, PASS); PreparedStatement
pstmt = conn.prepareStatement(query)) { pstmt.setInt(1, id);
    pstmt.setString(2, name);
    pstmt.setDouble(3, price);
    pstmt.executeUpdate();
    System.out.println("Product inserted successfully.");
} catch (SQLException e) {
    e.printStackTrace();
}
}

// Retrieves products
public void printAllProducts() {
    String query = "SELECT * FROM products";
    try (Connection conn = DriverManager.getConnection(URL, USER, PASS);
Statement stmt = conn.createStatement();
ResultSet rs = stmt.executeQuery(query)) {
        while (rs.next()) {
            System.out.println("ID: " + rs.getInt("id") +
                ", Name: " + rs.getString("name") + ", Price: " +
                    rs.getDouble("price"));
        }
    } catch (SQLException e) {
        e.printStackTrace();
    }
}
}

```

B) JDBC Architecture:

JDBC architecture is classified into two layers:

1. JDBC API: Provides database-independent connectivity classes and interfaces (java.sql package) such as Connection, Statement, and ResultSet.
2. JDBC Driver API: Handles database-specific translations. Vendor drivers (MySQL, Oracle) implement this layer to translate generic JDBC calls into native database protocol commands.

C) Steps Involved in Database Connectivity:

1. Load and Register the Driver: Load database vendor driver (e.g., Class.forName("com.mysql.cj.jdbc.Driver")).
2. Establish Connection: Call DriverManager.getConnection() with URL, user, and password parameters.
3. Create Statement: Create Connection.prepareStatement() or Connection.createStatement().
4. Execute Query: Call executeQuery() for SELECT queries, or executeUpdate() for INSERT/UPDATE/DELETE queries.

5. Process ResultSet: Iterate over ResultSet using next() to pull rows of data. 6. Close Resources: Close ResultSet, Statement, and Connection (accomplished cleanly using try with-resources).

**RUBRICS FOR CALCULATION:
Scenario-Based Assessment**

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand